Exercises.

① Regex accepting any lines that contain at least 3 vowels.

1) `[aeiouAEiOU].*[aeiouAEiOU].*[aeiouAEiOU]`

first vowel → anything → anything

2) `^(.*[aeiouAEiOU]){3}.*$`

start → anything → vowel → repeat 3 times → anything

② What printed?

```
char *s[3]={"A","B","C"};
for(int i=0; i<3; i++){
    execl("/bin/echo", "/bin/echo", s[i], NULL);
}
```

i=0 => s[i]="A" => /bin/echo A => success => stop (nothing else gets executed after successful exec());

③ Principle of locality regarding pages loading?

=> spatial => pages are near.

=> temporal => pages ~~are~~ recently used.

=> Load nearby pages when we load a file because it's likely we might need them soon.

④ Schedule jobs so delay is minimised.

A|5|7 B|2|4 C|4|13 D|3|8

Try ABCD: A: finishes at 5; deadline: 7 => 5 < 7 ✓

B: finishes at 5+2=7; deadline: 7 => 7 ≤ 7 ✓

C: finishes at 7+4=11; deadline: 13 => ✓

D: finishes at 11+3=14; deadline: ~~11~~ 8 => X.

Try BACD: B: finishes at 2: d: 7 ✓

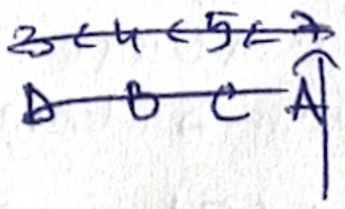
A: 2+5=7; d=7 ✓

C: 7+4=11; d=13 ✓

D: 11+3=14; d=8 X

EDD: BADC

4<7<8<13 ✓



Try BADC: ~~B: 2<4 ✓~~ B: 2<4, delay 0

~~A: 7,7 ✓~~ A: 2+5=7 ≤ 7, delay 0

~~D: 7+3=11~~ D: 7+3=10 > 8, delay 2.

C: 10+4=14 > 13, delay 1

=> 0+0+2+1=3. total delay

⑤ Risk brought by function f when executed in > threads?

⇒ DEADLOCK ; pthread_mutex_t m[2] ⇒ 2 threads. ⇒ m[0] / m[1]

id=0
first = m[0]
second = m[1]

→ lock(m[0]) ✓ ⇒ thread 0 locks m[0];
lock(m[1]) ← waits for m[1].

id=1
first = m[1]
second = m[0]

→ lock(m[1]) ✓ ⇒ thread 1 locks m[1].
lock(m[0]) ← waits for m[0]

⇒ deadlock. because mutexes are not locked in a consistent or

⑥ (+) and (−) of First Fit placement policy vs Worst Fit ....

(+) faster (does not search for the whole list)

(−) leaves small holes in the memory

⑦ Most prioritary page that the NRU replacement policy chooses as a victim page?

class 0: R=0, M=0 ✓ → if empty
1: R=0, M=1 first page from C1.
2: R=1, M=0
3: R=1, M=1.

⑧ size of block: B
size of address: A

How many data blocks are addressed by the triple indirect addressing of an inode?

$\left(\frac{B}{A}\right)^3$

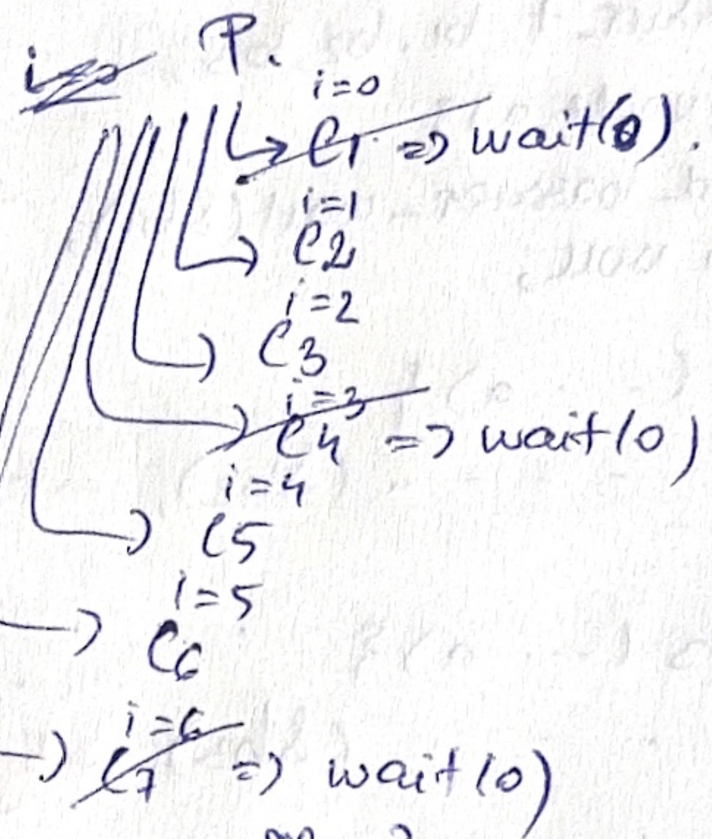
⑨ REGEX that accepts lines that contain "a", but not "b".

1) ^[^b]*a[^b]*$
^ start line; [^b]* 0 or more non-b characters; a at least one a

2) ^(?!.*b).*a.*$
^ start line; (?!.*b) must not contain b; .* anything; a at least one a; .* anything

(1) * => Max. nr. of child processes that can coexist sim.

```
for (int i=0; i<7; i++) {
    if (fork == 0) {
        sleep (rand () % 10);
        exit(0); => back to parent
    }
    if (i % 3 == 0)
        wait(0);
}
```



(2) (3) (4) (5) (6) before the last wait() was called, there were 5 active processes (max)

(11) How many threads for processing a million files?

- If CPU-bound => N K threads (K cores)
- If I/O-bound => > K threads. (because many threads will block while waiting for I/O).

=> start with K threads, then add more for I/O-bound workloads.
(nr. of CPU cores)

(12) Why does an I/O operation cause a process to move from running state to waiting state?

=> Process must wait for the I/O operation to complete before continuing the execution. While waiting, it is moved from RUN to WAIT so that the CPU can be used somewhere else.

(13) address calculation in the absolute fixed partition-allocation?

=> program is compiled for a specific memory partition => no translation

logical addr. = physical addr.

(14) A --X--> B B --Y--> C C --Z--> A Code for opening the FIFOs.

A.cpp
```
int a2b = open("X", O_WRONLY);
int c2a = open("Z", O_RDONLY);
```

B.cpp
```
int a2b = open("X", O_RDONLY);
int b2c = open("Y", O_WRONLY);
```

C.cpp
```
int c2a = open("Z", O_WRONLY);
int b2c = open("Y", O_RDONLY);
```

(15) T, N1, N2, N3 s.t. works:

5 5 5 5
3 3 3 3

```
pthread_barrier_t b1, b2, b3;
void* f1 (void* a){
    pthread_barrier_wait(&b1);   → type-1 threads wait at b1
    return NULL;
}
void* f2 (... a){
    ......(&b2);   → type-2 threads wait at b2
    ...
}
void* f3 (... a){
    ...(&b3);   → type-3 threads wait at b3.
}
int main(){
    int i;
    pthread_t t[T][3];   → T rows, 3 threads/row
    pthread_barrier_init(&b1, N1);   → b1 opens when N1 threads arrive
                ....(&b2, N2);   → b2   N2
                ....(&b3, N3);   → b3   N3
    for(..T){   ← repeat T times
        pthread_create(&t[i][0], NULL, f1, NULL);   → 1 thread for b1
                ...(  [i][1], ..., f2, ...)   → 1 thread for b2
                ...(  [i][2], ..., f3, ...)   → 1 thread for b3
    }
    for(...<T){   ← wait for all threads
        pthread_join(t[i][0], NULL);
                ...(  [i][1], ...);
                ...(  [i][2], ...);
    }
    pthread_barrier_destroy(&b1);
                ...(&b2);
                ...(&b3);
    return NULL;
}
```

⇒ waiting threads:
b1 : T
b2 : T
b3 : T

↓

N1, N2, N3 must divide T

1) nr. of f1 threads = nr. of f2 ... = nr. of f3 = T.
2) barrier releases N1 / N2 / N3

⇒ T % N1 = 0

∴ T = N1 = N2 = N3 = 3.

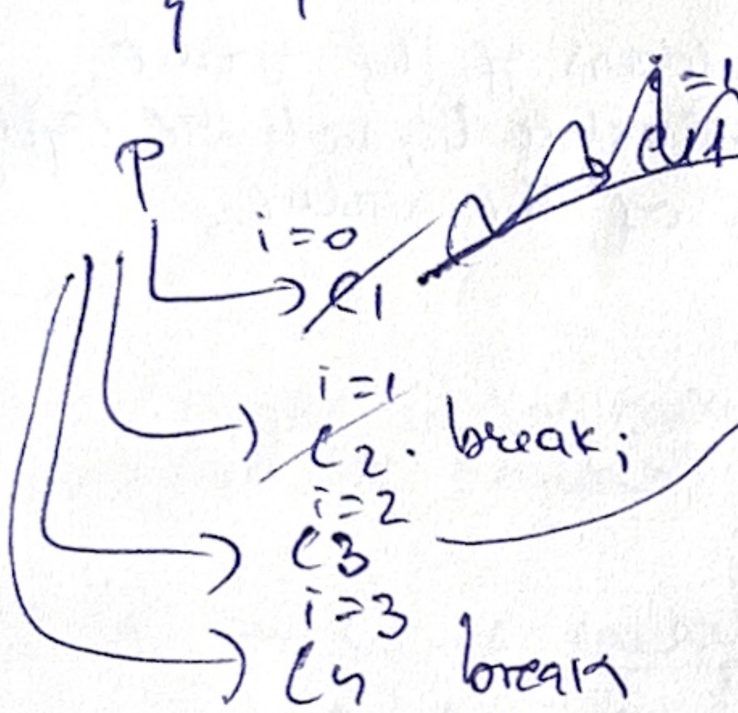
≠ val: T = 12, N1 = 2, N2 = 3, N3 = 4.

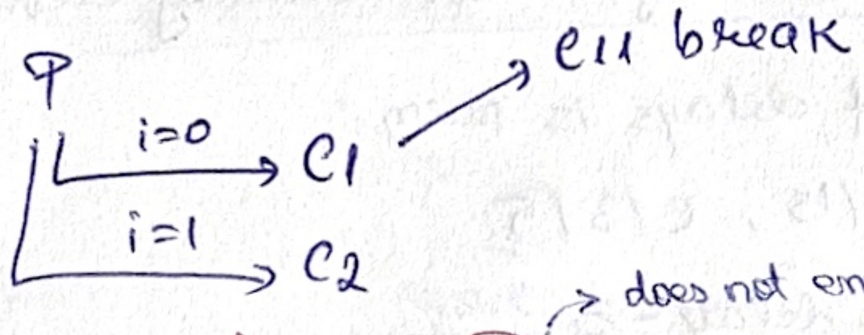
Explain!

① How many processes? (without P)

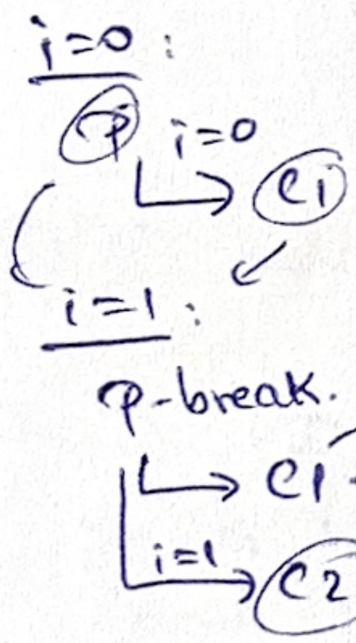
```
for (i=0; i<4; i++){
    if (fork() && i%2==1){
        break;
} }
```

TO REDO!





i=0:



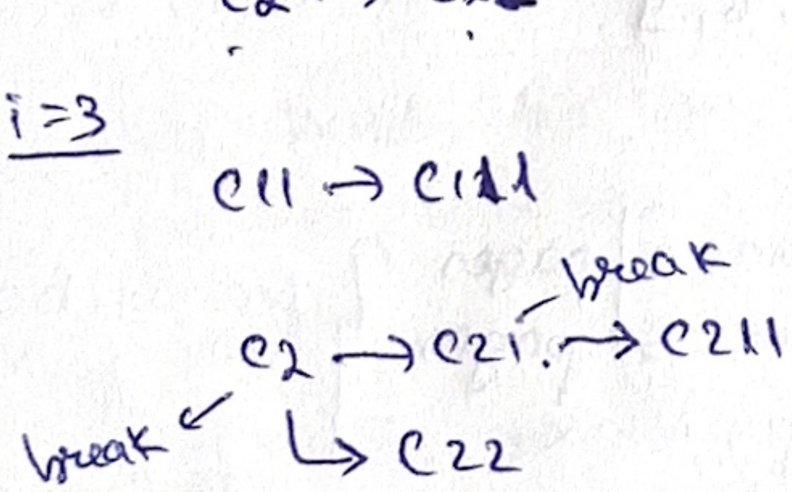
i=2

C11 → C111

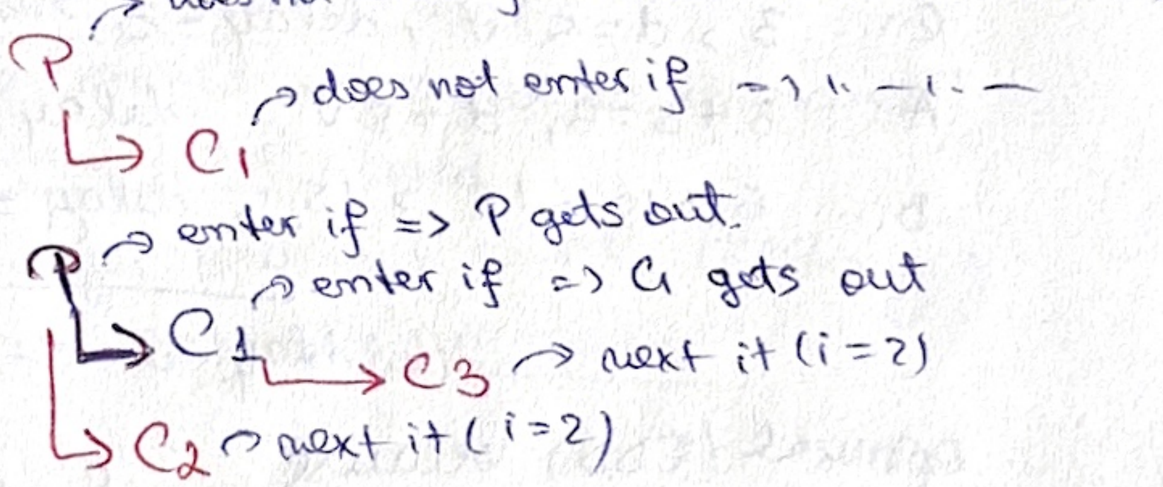
C2 → C21

i=3

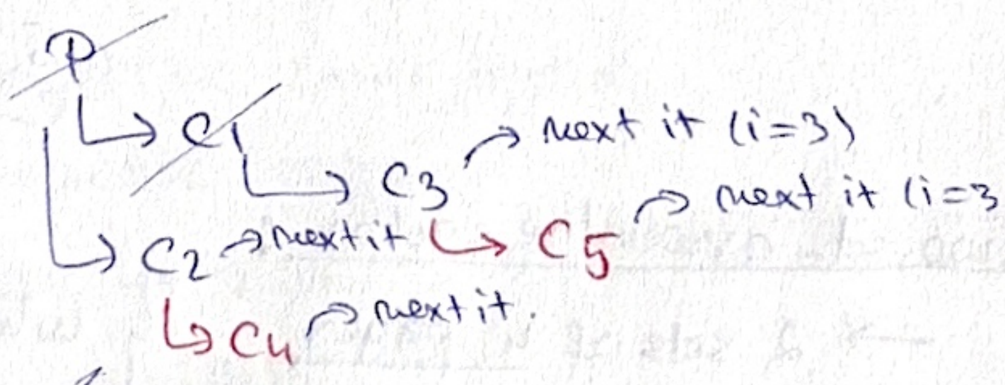
C11 → C111



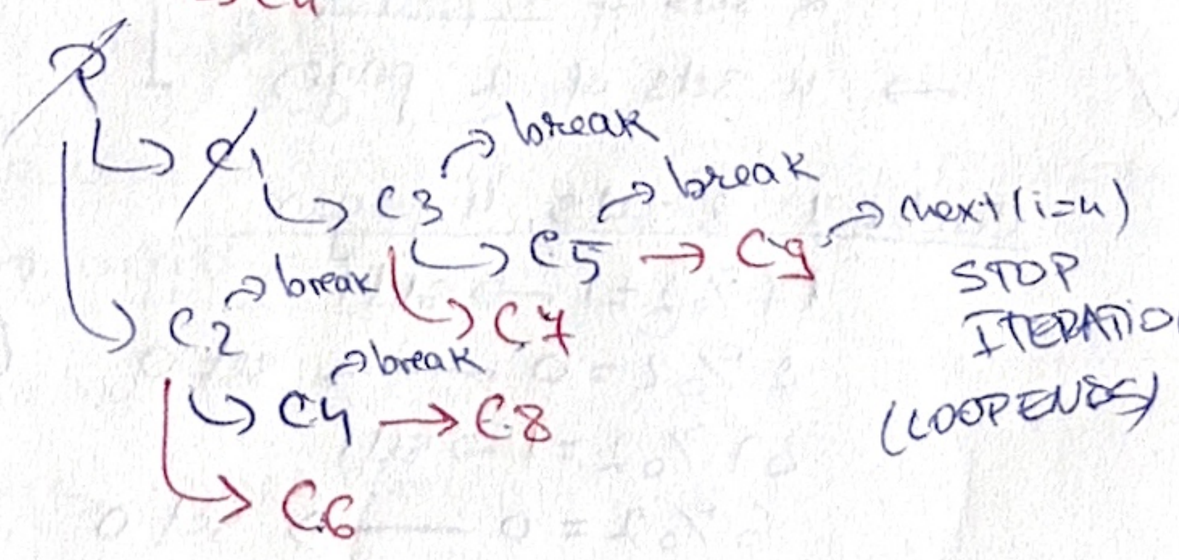
i=0



i=2



i=3



=> 9 children.

(17) What display?

```
void * f(void* a) {
    printf("%d\n", *(int*)a);   // address of i
    return NULL;
}
int main() {
    for (i=0; i<10; i++) {
        pthread_create(&t[i], NULL, f, &i);   // address of the same variable i, not the copy of the value.
    }
    return 0;
}
```

⇒ unpredictable ⇒ → 01...9
→ 3 3 7 10 10 ...
→ 10 10 10 ...

by the time (f) runs (i) might have already changed (still an issue)

(18) Schedule s.t. sum of delays is min.

A|3|8, B|10|15, C|3|5

C: 3, d=5 ✓, delay=0
A: 3+3=6, d=8 ✓, delay=0
B: 6+10=16 > 15, delay=1.

total delay: 0+0+1=1.

optimal (EDD) because C < A < B
5 8 15
deadlines

(19) Two set-associative caches;
→ 2 sets of 4 pages
→ 4 sets of 2 pages
} which would perform better?

17, 2, 37, 6, 9
↓
page-nr.

Cache 1: (2) sets, 4 pages

17 % 2 = 1 → set 1
2 % 2 = 0 → set 0
37 % 2 = 1 → set 1
6 % 2 = 0 → set 0
9 % 2 = 1 → set 1

⇒ set 0 has 2 pages
⇒ set 1 has 3 pages.
(4 pages max)
} no eviction ⇓ performs better.

Cache 2: (4) sets, 2 pages

17 % 4 = 1 → set 1
2 % 4 = 2 → set 2
37 % 4 = 1 → set 1
6 % 4 = 2 → set 2
9 % 4 = 1 → set 1

⇒ set 1 has 3 pages
⇒ set 2 has 2 pages
(2 pages max)
} ⇒ set 1 overflows ⇒ eviction needed.

block contains N addresses towards other blocks

✓ How many data blocks are addressed by an inode's double and triple indirections together?

$N^2 + N^3$.

(21) ✓ fixed partition allocation vs relocatable partition allocation

(+) faster/simpler address calculation (predefined RAM regions)

(−) poor memory utilisation due to fragmentation

(22) (+) and (−) of loading all the pages of a process in memory at once vs. loading them on demand.

(+) no page faults during execution

(−) more memory used (we might not need to access all)
more time used (longer startup until loading)

(23) (+) and (−) of direct access cache vs associative cache

(+) faster to search for a page

(−) no pages w same index => remove page

(24) When process changes state: WAIT → RUN ?

✓ => when event/condition becomes available

(25) Why hard-link can be created towards files on the same partition but not towards files on ≠partitions?

✓ hard link → inode (stores an inode nr.)
∃ inode (≠) ∈ same filesys./partition

(26) Add code so it works.

(26)

```
int p[2];
pipe(p);
dup2(p[1], 1);
printf("asdf\n");
fflush(stdout);
dup2(saved, 1);
close(saved);
printf("asdf\n");
```

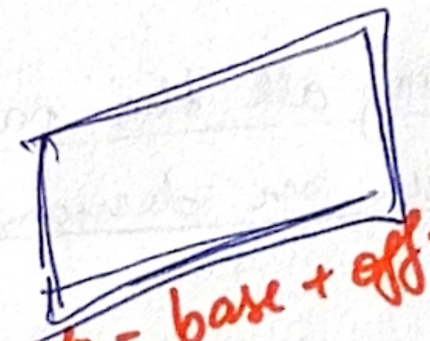
add code so it displays to console.

out → clone pipe's write end to fd 1 (stdout)

→ to make printf display to the console after dup2!

→ writes to a buffer.

```
int main()
{ int p[2];
  pipe(p);
  if (fork()==0)
  { close(p[1]);
    dup2(p[0], 0);
    ;
```



(27) elements of a virtual address in the page-segmented allocation.
what tables involved in calculating the physical addr.?

→ <segment, page, offset> → uses page nr. to find frame nr.

→ uses segment nr. to find the base addr. of that segment's page table.

→ tables involved: page table, segment table

(28) 4 REGEX that specify the nr. of times the previous expression should appear;

a* => 0 or more

a+ => 1 or more

? => 0 or 1

{x,y} => between x and y

(29) A process is opening a FIFO for writing
- the only one using that fifo.

When will the process finish the execution?

→ Never, because there is no one to read.